YILDIZ TECHNICAL UNIVERSITY APPLIED SQL (MTM4692) TECHNICAL REPORT

Hospital Information and Management System

Melisa Tamer 22058604 / Muharrem Mert Bayram 22058034

Team Number:7 / Github link: https://github.com/mertbayram1/HIMS-DATA-WAREHOUSE

1.Introduction

This project is a hospital management system that handles patient registration while allowing doctors to view full medical histories and add instant notes. It also enables them to track past diagnoses and treatments, and manage prescriptions digitally, facilitating fast and secure healthcare.

2.Problem Statement

Hospitals usually manage appointments, consultations, prescriptions, and invoices in separate systems. Because this data is not kept in one place, getting answers to basic management questions is slow and inconsistent. This project solves this problem by creating a single, SQLite-based data warehouse. Now, decision-makers can easily track department workloads, financial performance, medication stock risks, and patient revisit rates using standard SQL queries.

3.Importance of the Project

This project helps hospitals manage their operations more easily and effectively. It enables capacity planning by identifying peak hours and preventing departmental overload. On the financial side, administrators can easily monitor net revenue, collected payments, and outstanding balances. Also, the system ensures inventory safety through early alerts for low-stock medications. Beyond daily operations, it directly supports patient care quality by revisit rates.

4.Preliminary Schema & Relationships

The database consists of 9 tables following a star-schema design: 4 dimension tables, 4 fact

tables, and 1 operational table. Fact tables reference dimension tables via foreign keys.

Table	Type	Primary Key	Relationships & Notes
dim_patient	Dimension	patient_key	Books appointments, pays invoices
dim_doctor	Dimension	doctor_key	Employed by department, performs consultations
dim_department	Dimension	department_key	Hosts appointments, employs doctors
dim_medication	Dimension	medication_key	Used in prescriptions, triggers stock alerts
fact_appointment	Fact	appointment_key	Links patient, doctor & department; results in consultation
fact_consultation	Fact	consultation_key	1:1 with appointment; billed as invoice, includes prescriptions
fact_invoice	Fact	invoice_key	1:1 with consultation; tracks payments & net amount (Generated)
fact_prescription_detail	Fact	prescription_detail_key	Links consultation to medications with quantity & price
ops_stock_alert	Operational	alert_key	Auto-triggered when medication stock falls below critical level

5.Proposed SQL Features

- SQL Features Used in This Project:
- Table Design : PRIMARY KEY, FOREIGN KEY, UNIQUE, CHECK, DEFAULT, and computed columns (GENERATED)
- Indexes : CREATE INDEX to improve query performance
- Views : Analytical and summary views for reusable reports
- CTEs and Window Functions : WITH clauses and LEAD function for advanced analytics
- Conditional Aggregation : CASE WHEN ... THEN ... END inside SUM and COUNT
- Time Analytics : strftime and julianday for date, month, and hour calculations
- Triggers : BEFORE/AFTER INSERT, UPDATE, and DELETE triggers for automation and data integrity
- Error Handling : RAISE(ABORT) inside triggers to enforce business rules
- Automated Alerts : ops_stock_alert table and triggers for low-stock monito

6.Sample Queries

Sample 1: Patient Revisit Analysis

This query finds how many patients came back within 7 days after their appointment, using CTE and LEAD window function.

```sql
WITH ordered AS (
    SELECT patient_key,
           datetime(appointment_datetime) AS appointment_dt,
           LEAD(datetime(appointment_datetime)) OVER (
               PARTITION BY patient_key ORDER BY datetime(appointment_datetime)
           ) AS next_appointment_dt
    FROM fact_appointment
    WHERE appointment_status = 'COMPLETED'
)
SELECT COUNT(*) AS patient_count,
       ROUND(100.0 * SUM(CASE WHEN next_appointment_dt IS NOT NULL
                              AND julianday(next_appointment_dt) -
julianday(appointment_dt) <= 7
                         THEN 1 ELSE 0 END) / NULLIF(COUNT(*), 0), 2) AS
revisit_7d_rate_pct
FROM ordered;
```

Sample 2 : Department Occupancy

This query shows how busy each department is over the last 90 days, using JOIN and conditional aggregation.

```sql
SELECT d.department_name,
       COUNT(a.appointment_key) AS total_appointments_90d,
       ROUND(100.0 * SUM(CASE WHEN a.appointment_status = 'COMPLETED' THEN 1 ELSE 0
END)
             / NULLIF(COUNT(a.appointment_key), 0), 2) AS completion_rate_pct
FROM dim_department d
LEFT JOIN fact_appointment a ON a.department_key = d.department_key
  AND datetime(a.appointment_datetime) >= datetime('now', '-90 day')
GROUP BY d.department_name;
```

Sample 3: Invoice Trend

This query groups invoices by year and month to track revenue and payment rates over time, using strftime for date formatting.

```sql
SELECT CAST(strftime('%Y', invoice_date) AS INTEGER) AS invoice_year,
       CAST(strftime('%m', invoice_date) AS INTEGER) AS invoice_month,
       COUNT(*) AS invoice_count,
       ROUND(SUM(gross_amount), 2) AS gross_total,
       ROUND(100.0 * SUM(CASE WHEN payment_status = 'PAID' THEN 1 ELSE 0 END)
             / NULLIF(COUNT(*), 0), 2) AS paid_ratio_pct
FROM fact_invoice
GROUP BY CAST(strftime('%Y', invoice_date) AS INTEGER), CAST(strftime('%m',
invoice_date) AS INTEGER);
```

7.Conclusion

This hospital management system brings together patient registration, medical history tracking, and financial monitoring into a single, well-structured platform. By combining reliable data management with real-time analytics, it helps hospitals work more efficiently and deliver better patient care.